МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«ПОВОЛЖСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра ИиСП

Отчет

по лабораторной работе № 1

по дисциплине «Машинно-зависимые языки программирования»

Вариант 13

Выполнил: ст. гр. ПС-11

Вылегжанин Н.Ю.

Проверил: доцент, доцент кафедры ИиСП Баев А.А.

г. Йошкар-Ола

2026

Цель работы:

Восстановить ассемблерный код из hex-кода и освоить принципы дизассемблирования

Задания на лабораторную работу:

1. Провести процедуру восстановления ассемблерного кода по hex коду.
2. Вычислить длительность задержки
3. Построить алгоритм и написать код на С.
4. Перевести команды в ассемблерный листинг

1. Теоретические сведения

1. **Дизассемблирование** – это процесс восстановления ассемблерного кода из машинного кода. Оно позволяет понять как работает программа.

2. **Этапы дизассемблирования:**
   - Получение hex-строки
   - Выделение данных из кода
   - Перевод команд в little-endian
   - Определение команды и ее операндов
   - Формирование ассемблерного листинга

3. **Как работает дизассемблер?**
   Дизассемблер читает байты по порядку, определяет команду по таблице команд для определенного процессора и переходит к следующей.

   Существует два основных алгоритма:

   1. **Линейная развертка.** Использует прямолинейный подход: где завершается одна команда, начинается новая. Из за этого наиболее сложной задачей становится определение первой команды. Преимущество состоит в том, что этот алгоритм полностью покрывает кодовый сегмент, но так же он не может распознавать данные, совмещенные с кодом, что является главным недостатком.

   2. **Рекурсивного спуска.** По наличию ссылок на команду определяет должна она быть дизассемблирована или нет. Этот алгоритм отлично разделяет код и данные, но не может распознать ветви, образуемые такими командами, как jmp и call.

4. **Как различные архитектуры процессоров (например, x86 и ARM) влияют на процесс дизассемблирования?**
   x86 имеет переменный набор инструкций, что усложняет работу для дизассемблера.
   ARM уже имеет фиксированный размер инструкций, что, соответственно, упрощает работу.

5. **Как работает стек в ассемблере и как используются операции PUSH и POP?**
   Стек это область памяти, которая работает по принципу LIFO, что значит «последний зашел – первый вышел». Это значит, что последнее записанное значение выйдет из памяти первым.

   PUSH Rr записывает содержимое регистра Rr в стек. SP уменьшается на 1.
   POP Rd записывает байт из стека в регистр Rd. SP увеличивается на 1.

2. Практическая часть

F_CPU = 12000000UL (12.0 МГц)

```hex
:100000000C9434000C943E000C943E000C943E0082
:100010000C943E000C943E000C943E000C943E0068
:100020000C943E000C943E000C943E000C943E0058
:100030000C943E000C943E000C943E000C943E0048
:100040000C943E000C943E000C943E000C943E0038
:100050000C943E000C943E000C943E000C943E0028
:100060000C943E000C943E0011241FBECFEFD8E04C
:10007000DEBFCDBF0E9440000C9455000C940000E0
:10008000229A81E021E0982F9227813061F02A9A0C
:100090003FEF4AEA89E0315040408040E1F700C03C
:0E00A0000000892FF0CF2A98F3CFF894FFCFFD
:00000001FF
```

Первым делом разложим строку:
:10 0000 00 0C94 3400 0C94 3E00 0C94 3E00 0C94 3E00 82
:LL AAAA TT DDDDDDDDDDDDDDDDDDDDDDDDDDDDDD CC

LL – количество байт в поле данных
AAAA – 16-битное смещение начала адреса начала данных
TT – тип записи (00 – данные, 01 – конец файла, 04 – расширенный)
DDD… – непосредственно данные
CC – контрольная сумма

Выведем из этой строки данные и приведем в little-endian:
940C 0034 940C 003E 940C 003E 940C 003E

Переведем в двоичную систему счисления первую команду:
0C94 3400 -> 94 0C 00 34
1001 0100 0000 1100 0000 0000 0011 0100

Подберем маску к этому коду:
1001 010k kkkk 110k kkkk kkkk kkkk kkkk (jmp k)

K = 0 0000 0 0000 0000 0011 0100 = 0011 0100

Сместим адрес на один бит влево, т.к. адреса в этих командах (jmp, rjmp, call) хранятся в словах и чтобы перевести их в байты, нужно умножить результат на 2.
K = 0011 0100 << 1 (words -> bytes) = 0110 1000 = 0x68

Результат: 0: 0C 94 34 00 jmp 0x68 (Перемещение на адрес 0x68)

По той же аналогии разберем остальные hex строчки.

940C 003E -> 94 0C 00 3E
1001 0100 0000 1100 0000 0000 0011 1110
1001 010k kkkk 110k kkkk kkkk kkkk kkkk (jmp k)
K = 0 0000 0 0000 0000 0011 1110 = 0011 1110 << 1 (words -> bytes) = 0111 1100 = 0x7C
Результат: 4: 0C 94 3E 00 jmp 0x7C (Перемещение на адрес 0x7C)

Далее можем увидеть, что команды jmp 0x7C повторяются. Получаем:

8: 0C 94 3E 00 jmp 0x7C; 0x7C
C: 0C 94 3E 00 jmp 0x7C; 0x7C
10: 0C 94 3E 00 jmp 0x7C; 0x7C
14: 0C 94 3E 00 jmp 0x7C; 0x7C
18: 0C 94 3E 00 jmp 0x7C; 0x7C
1C: 0C 94 3E 00 jmp 0x7C; 0x7C
20: 0C 94 3E 00 jmp 0x7C 0x7C
24: 0C 94 3E 00 jmp 0x7C; 0x7C
28: 0C 94 3E 00 jmp 0x7C; 0x7C
2C: 0C 94 3E 00 jmp 0x7C; 0x7C
30: 0C 94 3E 00 jmp 0x7C; 0x7C
34: 0C 94 3E 00 jmp 0x7C; 0x7C
38: 0C 94 3E 00 jmp 0x7C; 0x7C
3C: 0C 94 3E 00 jmp 0x7C; 0x7C
40: 0C 94 3E 00 jmp 0x7C; 0x7C
44: 0C 94 3E 00 jmp 0x7C; 0x7C
48: 0C 94 3E 00 jmp 0x7C; 0x7C
4C: 0C 94 3E 00 jmp 0x7C; 0x7C
50: 0C 94 3E 00 jmp 0x7C; 0x7C
54: 0C 94 3E 00 jmp 0x7C; 0x7C
58: 0C 94 3E 00 jmp 0x7C; 0x7C
5C: 0C 94 3E 00 jmp 0x7C; 0x7C
60: 0C 94 3E 00 jmp 0x7C; 0x7C
64: 0C 94 3E 00 jmp 0x7C; 0x7C

Строка 0060

:10 0060 00 0C94 3E00 0C94 3E00 1124 1FBE CFEF D8E0 4C
~~940C 003E 940C 003E~~ 2411 BE1F EFCE E0D8

1124 -> 24 11
0010 0100 0001 0001
0010 01rd dddd rrrr (eor *Rd*, *Rr*)
Rr = 0 0001 = 0x01 = r1
Rd = 0 0001 = 0x01 = r1
Результат: 68: 11 24 eor r1, r1 (Исключающее ИЛИ между регистрами r1 и r1, помещает результат в r1 (в данном случае регистр очистится))

1FBE -> BE 1F
1011 1110 0001 1111
1011 1AAr rrrr AAAA (out *A*, *Rr*)
A = 0011 1111 = 0x3F
Rr = 0 0001 = 0x01 = r1
Результат: 6A: 1F BE out 0x3F, r1; SREG (Сохраняет данные из регистра r1 в пространство ввода-вывода).
SREG это регистр состояния, который содержит биты флагов (I, T, H, S, V, N, Z, C).

CFEF -> EF CF
1110 1111 1100 1111
1110 KKKK dddd KKKK (ldi *Rd*, *K*)
K = 1111 1111 = 0xFF
В ldi регистр принимает значения в диапазоне от 16 до 31. Следовательно, чтобы получить нужный регистр, нужно к результату из маски (1100 = 12) прибавить 16:
Rd = 1100 = 16 + 12 = r28
Результат: 6C: CF EF ldi r28, 0xFF (0xFF записывается в регистр r28)

D8E0 -> E0 D8
1110 0000 1101 1000
1110 KKKK dddd KKKK (ldi *Rd*, *K*)
K = 0000 1000 = 0x08
Rd = 1101 = 16 + 13 = r29
Результат: 6E: D8 E0 ldi r29, 0x08 (0x08 записывается в регистр r29)

Строка 0070

:10 0070 00 DEBF CDBF 0E94 4000 0C94 5500 0C94 0000 E0
BFDE BFCD 940E 0040 940C 0055 940C 0000

DEBF -> BF DE
1011 1111 1101 1110
1011 1AAr rrrr AAAA (out *A*, *Rr*)
A = 0011 1110 = 0x3E
Rr = 0001 1101 = r29
Результат: 70: DE BF out 0x3E, r29; SPH (Сохраняет данные из регистра r29 в пространство ввода-вывода). SPH хранит старший бит указателя стека.

CDBF -> BF CD
1011 1111 1100 1101
1011 1AAr rrrr AAAA (out *A*, *Rr*)
A = 11 1101 = 0x3D
Rr = 1 1100 = r28
Результат: 72: CD BF out 0x3D, r28; SPL (Сохраняет данные из регистра r29 в пространство ввода-вывода). SPL хранит младший бит указателя стека.

0E94 4000 -> 94 0E 00 40
1001 0100 0000 1110 0100 0000 0000 0000
1001 010k kkkk 111k kkkk kkkk kkkk kkkk (call *k*)
K = 0 0000 0 0100 0000 0000 = 0100 0000 << 1 (words -> bytes) = 1000 0000 = 0x80
Результат: 74: 0E 94 40 00 call 0x80 (Вызывает подпрограмму в пределах всей программной памяти)

0C94 5500 -> 94 0C 00 55
1001 0100 0000 1100 0000 0000 0101 0101
1001 010k kkkk 110k kkkk kkkk kkkk kkkk (jmp *k*)
K = 0 0000 0 0000 0000 0101 0101 << 1 (words -> bytes) = 1010 1010= 0xAA
Результат: 78: 0C 94 55 00 jmp 0xAA (Перемещение на адрес 0xAA)

0C94 0000 -> 94 0C 00 00
1001 0100 0000 1100 0000 0000 0000 0000
1001 010k kkkk 110k kkkk kkkk kkkk kkkk (jmp *k*)
K = 0 0000 0 0000 0000 0000 0000 = 0x00
Результат: 7C: 0C 94 00 00 jmp 0x00 (Перемещение на адрес 0x00)

Строка 0080

:10 0080 00 229A 81E0 21E0 982F 9227 8130 61F0 2A9A 0C
9A22 E081 E021 2F98 2792 3081 F061 9A2A

229A -> 9A 22
1001 1010 0010 0010
1001 1010 AAAA Abbb (sbi *A*, *b*)
A = 0 0100 = 0x04
b = 0010 = 2
Результат: 80: 22 9A sbi 0x04, 2; DDRB (Устанавливает указанный бит в регистре ввода-вывода. В регистр DDRB установится бит 2). DDRB2 настраивает PC2 как выход.

81E0 -> E0 81
1110 0000 1000 0001
1110 KKKK dddd KKKK (ldi *Rd*, *K*)
K = 0000 0001 = 0x01
Rd = 1000 = 16 + 8 = r24
Результат: 82: 81 E0 ldi r24, 0x01 (0x01 записывается в регистр r24)

21E0 -> E0 21
1110 0000 0010 0001
1110 KKKK dddd KKKK (ldi *Rd*, *K*)
K = 0000 0001 = 0x01
Rd = 0010 = 16 + 2 = 18
Результат: 84: 21 E0 ldi r18, 0x01 (0x01 записывается в регистр r18)

982F -> 2F 98
0010 1111 1001 1000
0010 11rd dddd rrrr (mov *Rd*, *Rr*)
Rr = 0001 1000 = r24
Rd = 0001 1001 = r25
Результат: 86: 98 2F mov r25, r24 (Делает копию регистра r24 в регистр r25)

9227 -> 27 92
0010 0111 1001 0010
0010 01rd dddd rrrr (eor *Rd*, *Rr*)
Rr = 0001 0010 = r18
Rd = 0001 1001 = r25
Результат: 88: 92 27 eor r25, r18 (Исключающее ИЛИ между регистрами r25 и r18, помещает результат в r25)

8130 -> 30 81
0011 0000 1000 0001 (30 81)
0011 KKKK dddd KKKK (cpi *Rd*, *K*)
K = 0000 0001 = 0x01
Rd = 1000 = 16 + 8 = r24
Результат: 8A: 81 30 cpi r24, 0x01 (Выполняет сравнение между регистром r24 и константой 0x01, меняет флаги, не меняя значение регистра)

61F0 -> F0 61
1111 0000 0110 0001 (F0 61)
1111 00kk kkkk k001 (breq *k*)
K = 0001100 << 1 (words -> bytes) = 0011000= +24
Теперь нужно найти куда программа перейдет при выполнении условия. Вычислим по формуле:
PC = PC + k + 1
PC = 0x8C + 24 + 2 = 140 + 24 + 2 = 166 = 0xA6
Результат: 8C: 61 F0 breq +24 (Условный относительный переход. Проверяет флаг Zero (Z) и выполняет переход по адресу 0xA6, если Z=1)

2A9A -> 9A 2A
1001 1010 0010 1010
1001 1010 AAAA Abbb (sbi *A*, *b*)
A = 0 0101 = 0x05
b = 010 = 2
Результат: 8E: 2A 9A sbi 0x05, 2; PORTB (Устанавливает указанный бит в регистре ввода-вывода. В регистр PORTB установится бит 2). PORTB это выходные уровни.

Строка 0090

:10 0090 00 3FEF 4AEA 89E0 3150 4040 8040 E1F7 00C0 3C
FE3F EA4A E089 5031 4040 4080 F7E1 C000

3FFE -> FE 3F
1110 1111 0011 1111
1110 KKKK dddd KKKK (ldi *Rd*, *K*)
K = 1111 1111 = 0xFF
Rd = 0011 = 16 + 3 = r19
Результат: 90: 3F EF ldi 0xFF, r19 (0xFF загружается в регистр r19)

4AEA -> EA 4A
1110 1010 0100 1010
1110 KKKK dddd KKKK (ldi *Rd*, *K*)
K = 1010 1010 = 0xAA
Rd = 0100 = 16 + 4 = r20
Результат: 92: 4A EA ldi 0xAA, r20 (0xAA загружается в регистр r20)

89E0 -> E0 89
1110 0000 1000 1001
1110 KKKK dddd KKKK (ldi *Rd*, *K*)
K = 0000 1001 = 0x09
Rd = 1000 = 16 + 8 = r24
Результат: 94: 89 E0 ldi 0x09, r24 (0x09 загружается в регистр r24)

3150 -> 50 31
0101 0000 0011 0001
0101 KKKK dddd KKKK (subi *Rd*, *K*)
K = 0000 0001 = 0x01
Rd = 0011 = 16 + 3 = r19
Результат: 96: 31 50 subi r19, 0x01 (Вычитает регистр r19 и константу 0x01 и помещает результат в регистр назначения r19)

4040 -> 40 40
0100 0000 0100 0000
0100 KKKK dddd KKKK (sbci *Rd*, *K*)
K = 0000 0000 = 0x00
Rd = 0100 = 16 + 4 = r20
Результат: 98: 40 40 sbci r20, 0x00 (Вычитает константу 0x00 из регистра r20 и дополнительно вычитает флаг переноса C, помещая результат в регистр назначения r20)

8040 -> 40 80
0100 0000 1000 0000
0100 KKKK dddd KKKK (sbci *Rd*, *K*)
K = 0x00
Rd = 1000 = 16 + 8 = r24
Результат: 9A: 80 40 sbci r24, 0x00 (Вычитает константу 0x00 из регистра r24 и дополнительно вычитает флаг переноса C, помещая результат в регистр назначения r24)

E1F7 -> F7 E1
1111 0111 1110 0001
1111 01kk kkkk k001 (brne *k*)
K = 11 1110 0
Старшим битом является 1, значит число отрицательное. Инвертируем двоичный код, прибавляем к результату 1 и смещаем его влево на 1 бит (переводим из слов в байты)
K = -00 0001 1 + 1 = -0100 << 1 = -1000 = -8
PC = PC + k + 1
PC = 0x9c – 8 + 2 = 156 – 8 + 2 = 150 = 0x96
Результат: 9C: E1 F7 brne -8 (Условный относительный переход. Проверяет флаг Zero (Z) и выполняет переход по адресу 0x96, если Z=0)

00C0 -> C0 00
1100 0000 0000 0000 (C0 00)
1100 kkkk kkkk kkkk (rjmp *k*)
K = 0000 0000 0000 = 0000
Рассчитаем адрес для перехода:
PC = PC + k + 1
PC = 0x9e – 0 + 2 = 158 + 2 = 160 = 0xA0
Результат: 9E: 00 C0 rjmp +0, 0xA0 (Относительный переход на адрес 0xA0)

Строка 00A0

:0E 00A0 00 0000 892F F0CF 2A98 F3CF F894 FFCF FD
0000 2F89 CFF0 982A CFF3 94F8 CFFF

0000 0000 0000 0000 (nop)
Результат: A0: 00 00 nop

892F -> 2F 89
0010 1111 1000 1001
0010 11rd dddd rrrr (mov *Rd*, *Rr*)
Rr = 1 1001 = r25
Rd = 1 1000 = r24
Результат: A2: 89 2F mov r24, r25 (Делает копию регистра r25 в регистр r24)

F0CF -> CF F0
1100 1111 1111 0000
1100 kkkk kkkk kkkk (rjmp *k*)
K = 1111 1111 0000 = -0000 0000 1111 + 1 = -0001 0000 << 1 (words -> bytes)
= - 0010 0000 = -32
PC = PC + k + 1
PC = 0xA4 – 32 + 2= 0xA4 – 0x20 + 0x02 = 164 – 32 + 2 = 134 = 0x86
Результат: A4: F0 CF rjmp -32, 0x86 (Относительный переход на адрес 0x86)

2A98 -> 98 2A
1001 1000 0010 1010
1001 1000 AAAA Abbb (cbi *A*, *b*)
A = 00101 = 0x05
b = 0010 = 2
Результат: A6: 2A 98 cbi 0x05, 2; PORTB (Очищает определённый бит в регистре ввода-вывода. В регистре DDRB сбросится бит 2)

F3CF -> CF F3
1100 1111 1111 0011
1100 kkkk kkkk kkkk (rjmp *k*)
K = 1111 1111 0011 = - 0000 0000 1100 + 1 = - 1101 << 1 (words -> bytes) = - 0001 1010 = -26
PC = PC + k + 1
PC = 0xA8 – 26 + 2 = 0xA8 – 0x24 = 0x90
Результат: A8: F3 CF rjmp -26 0x90 (Относительный переход на адрес 0x90)

F894 -> 94 F8
1001 0100 1111 1000 (94 F8)
1001 0100 1111 1000 (cli)
Результат: AA: F8 94 cli (Команда запрещает прерывание)

FFCF -> CF FF
1100 1111 1111 1111 (CF FF)
1100 kkkk kkkk kkkk (rjmp *k*)
K = 1111 1111 1111 = - 0000 0000 0000 + 1 << 1 = -0010 (words -> bytes) = -2
PC = PC + k + 1
PC = 0xAC – 2 + 2 = 0xAC
Результат: AC: FF CF rjmp -2 0xAC (Относительный переход на адрес 0xAC)

ASM-листинг:

```
0: 0C 94 34 00 jmp 0x68; 0x68
4: 0C 94 3E 00 jmp 0x7C; 0x7C
8: 0C 94 3E 00 jmp 0x7C; 0x7C
C: 0C 94 3E 00 jmp 0x7C; 0x7C
10: 0C 94 3E 00 jmp 0x7C; 0x7C
14: 0C 94 3E 00 jmp 0x7C; 0x7C
18: 0C 94 3E 00 jmp 0x7C; 0x7C
1C: 0C 94 3E 00 jmp 0x7C; 0x7C
20: 0C 94 3E 00 jmp 0x7C; 0x7C
24: 0C 94 3E 00 jmp 0x7C; 0x7C
28: 0C 94 3E 00 jmp 0x7C; 0x7C
2C: 0C 94 3E 00 jmp 0x7C; 0x7C
30: 0C 94 3E 00 jmp 0x7Cc; 0x7C
34: 0C 94 3E 00 jmp 0x7C; 0x7C
38: 0C 94 3E 00 jmp 0x7C; 0x7C
3C: 0C 94 3E 00 jmp 0x7C; 0x7C
40: 0C 94 3E 00 jmp 0x7C; 0x7C
44: 0C 94 3E 00 jmp 0x7C; 0x7C
48: 0C 94 3E 00 jmp 0x7C; 0x7C
4C: 0C 94 3E 00 jmp 0x7C; 0x7C
50: 0C 94 3E 00 jmp 0x7C; 0x7C
54: 0C 94 3E 00 jmp 0x7C; 0x7C
58: 0C 94 3E 00 jmp 0x7C; 0x7C
5C: 0C 94 3E 00 jmp 0x7C; 0x7C
60: 0C 94 3E 00 jmp 0x7C; 0x7C
64: 0C 94 3E 00 jmp 0x7C; 0x7C
68: 11 24 eor r1, r1;
6A: 1F BE out 0x3F, r1; SREG
6C: CF EF ldi r28, 0xFF; 255
6E: D8 E0 ldi r29, 0x08; 8
70: DE BF out 0x3E, r29; SPH
72: CD BF out 0x3D, r28; SPL
74: 0E 94 40 00 call 0x80; 0x80
78: 0C 94 55 00 jmp 0xAA; 0xAA
7C: 0C 94 00 00 jmp 0x00; 0x00
80: 22 9A sbi 0x04, 2; DDRB
82: 81 E0 ldi r24, 0x01; 1
84: 21 E0 ldi r18, 0x01; 1
86: 98 2F mov r25, r24;
88: 92 27 eor r25, r18;
8A: 81 30 cpi r24, 0x01; 1
8C: 61 F0 breq +24; 0xA6
8E: 2A 9A sbi 0x05, 2; PORTB
90: 3F EF ldi r19, 0xFF; 255
```

92: 4A EA ldi r20, 0xAA; 170
94: 89 E0 ldi r24, 0x09; 9
96: 31 50 subi r19, 0x01; 1
98: 40 40 sbci r20, 0x00; 0
9A: 80 40 sbci r24, 0x00; 0
9C: E1 F7 brne -8; 0x96
9E: 00 C0 rjmp +0; 0xA0
A0: 00 00 nop;
A2: 89 2F mov r24, r25;
A4: F0 CF rjmp -32; 0x86
A6: 2A 98 cbi 0x05, 2; PORTB
A8: F3 CF rjmp -26; 0x90
AA: F8 94 cli;
AC: FF CF rjmp -2; 0xAC

Восстановление условного оператора и внешнего цикла

```
r25 = r25 ^ r18;                    // 88: 92 27 eor r25, r18;

if (r24 == 1)                       // 8A: 81 30 cpi r24, 0x01; 8C: 61 F0 breq +24; -> 0xA6
        PORTB &= ~(1 << PINB2)      // A6: 2A 98 cbi 0x05, 2; PORTB
else
        PORTB |= (1 << PINB2)       // 8E: 2A 9A sbi 0x05, 2; PORTB
```

Вычисление задержки
F_CPU = 12000000UL (12.0 МГц)

Команды 0x90-0x94 (включительно) записывают значения:
r19 = 0xFF
r20 = 0xAA
r24 = 0x09

Возьмем одно 24-разрядное число:
N = 0x09AAFF = 633 599

Участок SUBI, SBCI, SBCI, BRNE требует 5 циклов, следовательно:
5 * N – 1 = 5 * 633 599 – 1 = 3 167 994
С дополнительными командами получается 3 168 000
t = 3 168 000 / 12 000 000 = 264мс

Эквивалент на С

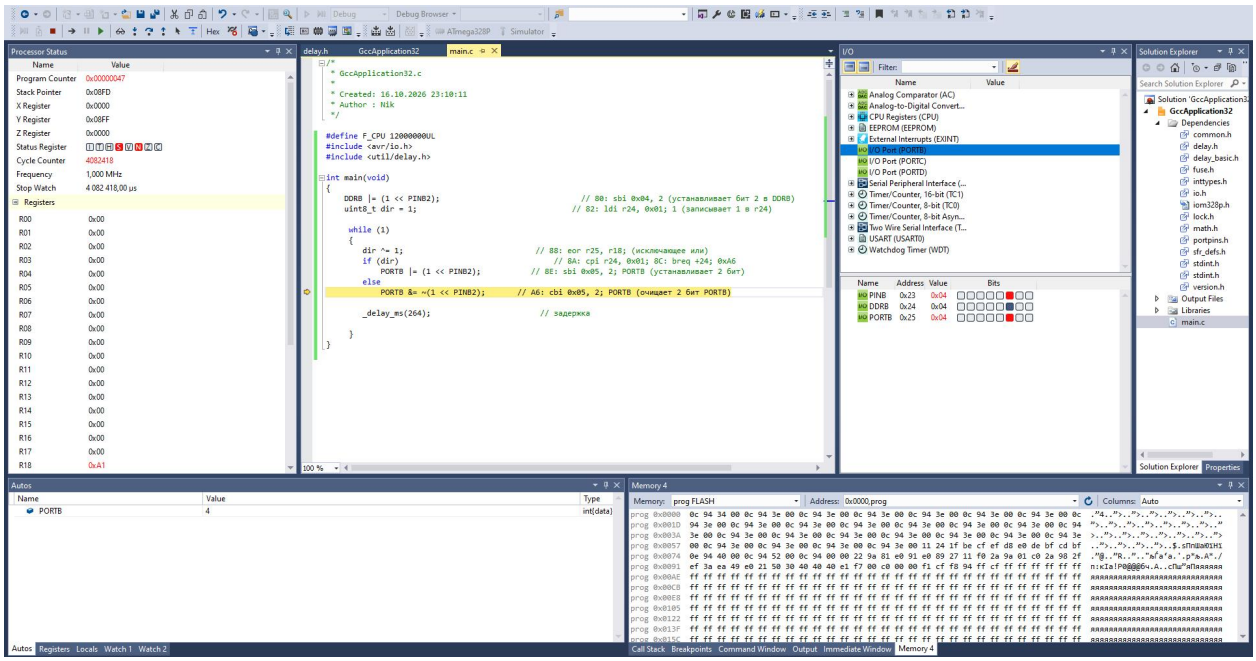
```
#define F_CPU 12000000UL
#include <avr/io.h>
#include <util/delay.h>

int main(void)
{
	DDRB |= (1 << PINB2);                    // 80: sbi 0x04, 2 (устанавливает бит 2 в DDRB)
	uint8_t dir = 1;                         // 82: ldi r24, 0x01; 1 (записывает 1 в r24)

	while (1)
	{
		dir ^= 1;                        // 88: eor r25, r18; (исключающее или)
		if (dir)                         // 8A: cpi r24, 0x01; 8C: breq +24; 0xA6
			PORTB |= (1 << PINB2);   // 8E: sbi 0x05, 2; PORTB (устанавливает 2 бит)
		else
			PORTB &= ~(1 << PINB2);  // A6: cbi 0x05, 2; PORTB (очищает 2 бит PORTB)

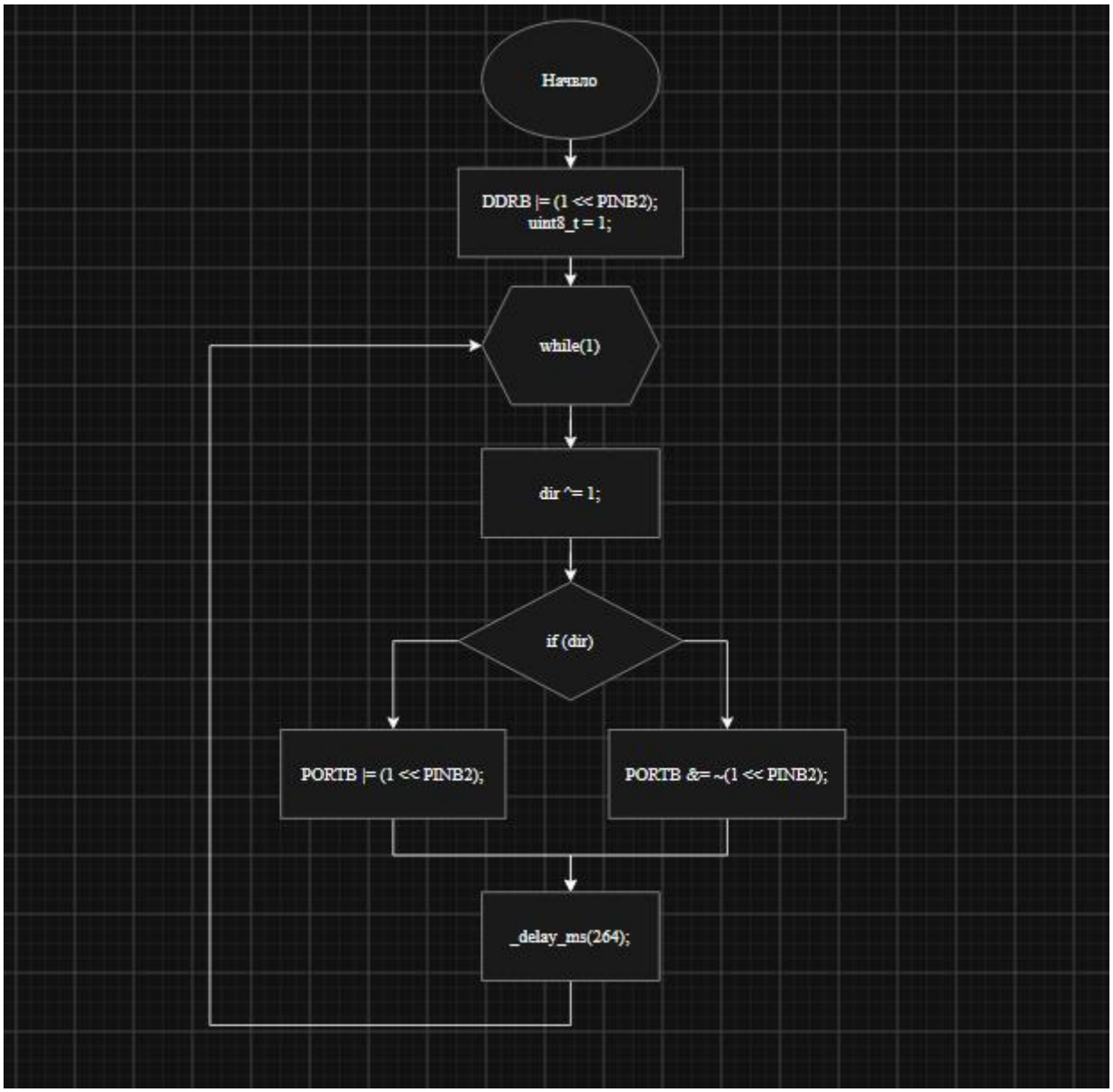
		_delay_ms(264);                  // задержка

	}
}
```



В PORTB видим 4 (во 2 бите установлено значение 1), предполагаемый светодиод зажегся.

Блок-схема С:


Начало
DDRB |= (1 << PINB2);
uint8_t = 1;
while(1)
dir ^= 1;
if (dir)
PORTB |= (1 << PINB2);
PORTB &= ~(1 << PINB2);
_delay_ms(264);

Ассемблерный код

```
reset:
   rjmp main            // переход в main (0x80)

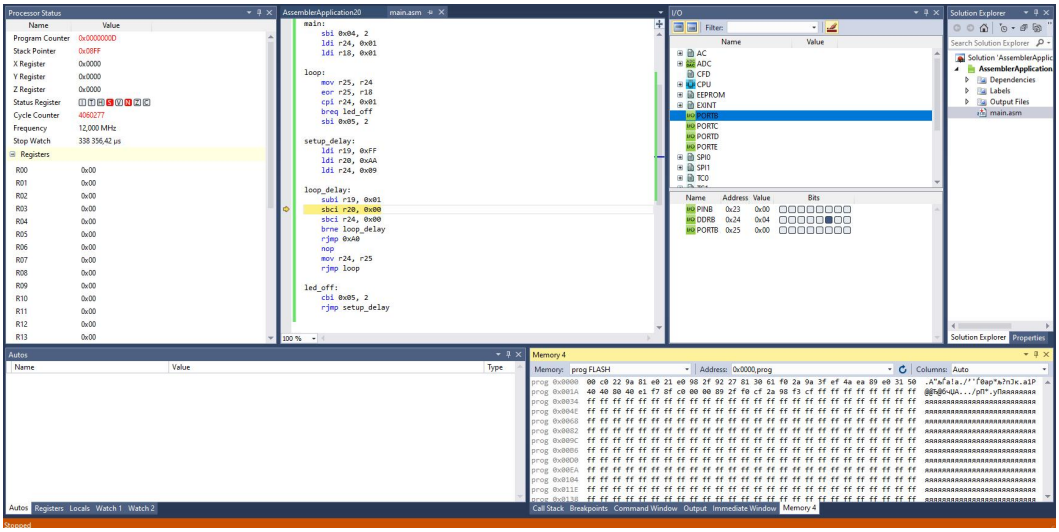
main:
   sbi DDRB, 2          // PB2 на выход
   ldi r24, 0x01        // r24 = 1
   ldi r18, 0x01        // r18 = 1

loop:
   mov r25, r24         // r25 = r24
   eor r25, r18         // r25 = r25 ^ r18
   cpi r24, 0x01        // условие if r24 == 1
   breq led_off         // переход в led_off (0xA6) если cpi выполняется
   sbi PORTB, 2         // устанавливает бит 2 в PORTB

setup_delay:
   ldi r19, 0xFF        // r19 = 0xFF
   ldi r20, 0xAA        // r20 = 0xAA
   ldi r24, 0x09        // r24 = 0x09

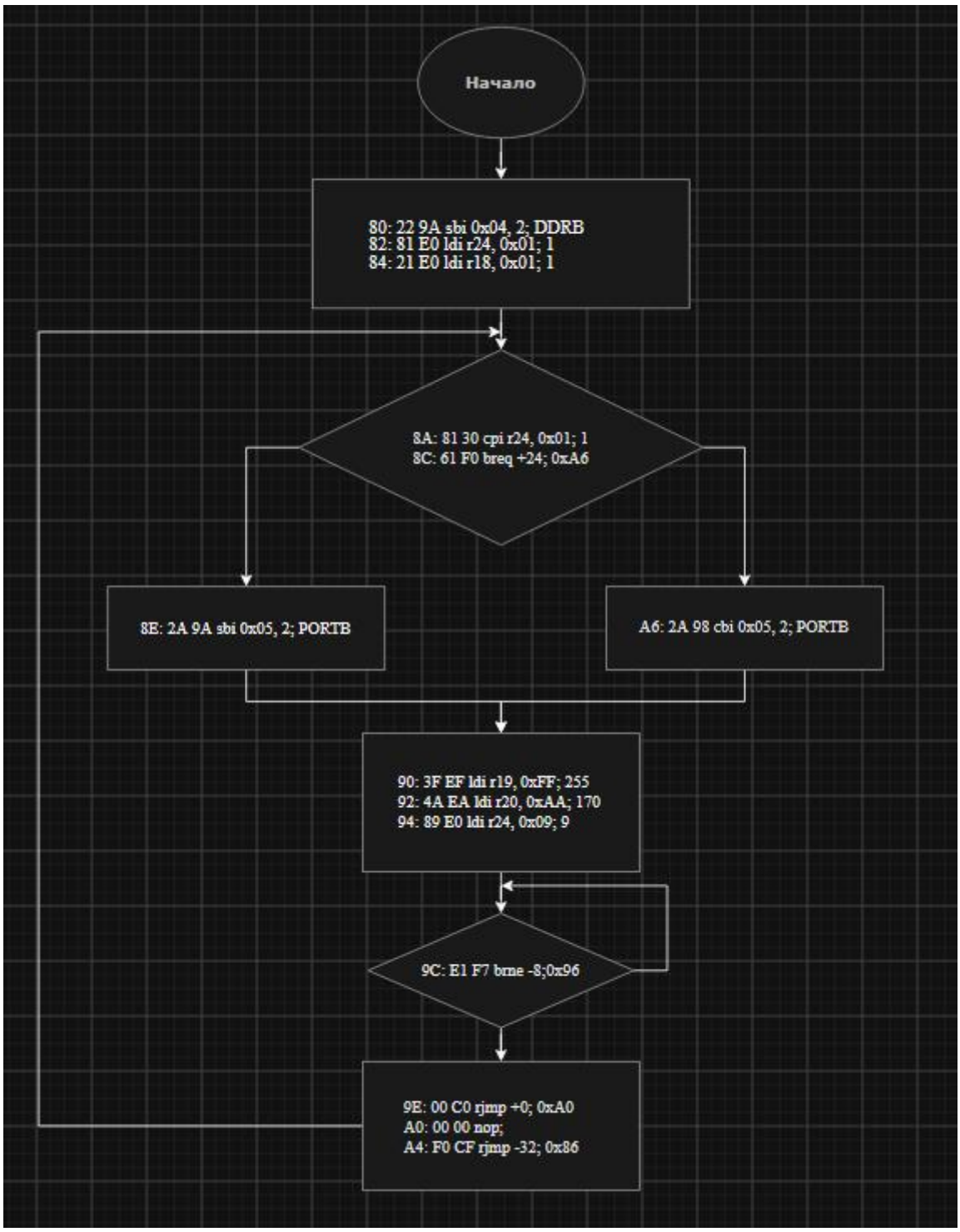
loop_delay:
   subi r19, 0x01       // r19 = 1
   sbci r20, 0x00       // r20 = 0
   sbci r24, 0x00       // r24 = 0
   brne loop_delay      // если Z=0, переход на loop_delay (0x96)
   rjmp 0xA0            // переход в 0xA0
   nop                  // ничего
   mov r24, r25         // r24 = r25
   rjmp loop            // переход в loop (0x86)

led_off:
   cbi PORTB, 2         // очищает бит 2 в PORTB
   rjmp setup_delay     // переход в setup_delay (0x90)
```



В PORTB уже 0 (во 2 бите значение 0). Значит диод погас.

Блок-схема ASM:



Выводы

Программа восстановлена, из получившегося кода получилась так называемая «мигалка», которая заставляет лампочку мигать с задержкой примерно 264мс.